# IoT Cat Feeder

An automatic, portion-controlled cat feeder with a web dashboard on the ESP8266

## Project Report

Chindanai Jaiman  662115012
Bachelor of Science Software Engineering Program
College of Arts, Media, and Technology, Chiang Mai University

November 2026

Project Advisor
**[Advisor Name]**

# Table of Contents

**Part I Project Proposal** ..... **4**
**Chapter 1 Introduction** ..... **5**
1.1 Motivation ..... 5
1.2 Purpose ..... 5
1.3 Target Users and Intended Experience ..... 5
1.4 Project Scope ..... 6
1.5 Objectives ..... 6
**Chapter 2 Review and Analysis** ..... **7**
2.1 Business and Software Review ..... 7
2.2 Tool and Technology Review ..... 7
2.3 Risk Analysis ..... 8
**Chapter 3 Product Perspective** ..... **9**
3.1 Product Perspective ..... 9
3.2 Product Features ..... 9
3.3 System Architecture ..... 9
3.4 Use Case Overview ..... 9
**Chapter 4 Quality Planning** ..... **11**
**Chapter 5 Limits, Milestones, and Deliverables** ..... **12**
5.1 Limits ..... 12
5.2 Milestones ..... 12
5.3 Deliverable Items ..... 12
5.4 Budget ..... 12
5.5 Future Work ..... 13
**Part II Software Requirements Specification** ..... **14**
**Chapter 6 Introduction** ..... **15**
6.1 Purpose ..... 15
6.2 Scope ..... 15
6.3 Product Perspective ..... 15
6.4 Product Functions ..... 16
6.5 User Characteristics ..... 16
6.6 Limitations ..... 16
6.7 Assumptions and Dependencies ..... 16
6.8 Acronyms and Definitions ..... 17
6.9 References ..... 17
6.10 Document Control ..... 18
**Chapter 7 Specification Requirements** ..... **19**
7.1 Feature Description ..... 19
7.2 Use Case Diagram ..... 20
7.3 Use Case Description ..... 21
7.4 Activity Diagram ..... 23
7.5 Functional Requirements ..... 23

7.6 External Interface Requirements....24
7.7 Data Requirements....25
7.8 Quality Requirements....26
7.9 Design Constraints....26
7.10 Verification....26
**Appendix A Rubric Coverage....28**
**Part III Software Design Description....29**
**Chapter 8 Introduction....30**
8.1 Purpose....30
8.2 Scope....30
8.3 Design Overview....30
8.4 Definitions....30
**Chapter 9 System Architecture....31**
9.1 Architectural Overview....31
9.2 Context View....31
9.3 Component View....32
9.4 Deployment and Power Design....33
9.5 Design Rationale....33
**Chapter 10 Detailed Design....34**
10.1 Data Design....34
10.2 Component Description....34
10.3 State Design....35
10.4 Interaction Flows....36
10.5 Interface Design....36
10.6 Timing and Resource Budget....37
10.7 Error Handling and Recovery....37
10.8 Security Considerations....38
**Chapter 11 Traceability and Verification....39**
11.1 Requirements Traceability Matrix....39
11.2 Test Plan....39
11.3 Known Limitations....39
11.4 Future Work....40
**Appendix B Design Review Checklist....41**

# Part I Project Proposal

*Problem, objectives, scope, review, and quality planning.*

# Chapter 1 Introduction

## 1.1 Motivation

A cat that is fed by hand eats at the convenience of the owner, not at a fixed time. When the owner leaves for work in the morning and returns in the evening, the usual workaround is to pour a large portion into the bowl and let the cat graze all day. Two problems follow. The food sits in the open bowl for ten hours or more, and the cat eats the entire portion before noon and then begs for the rest of the day.

The second problem is portion control. Dry food is calorie dense, and a cat that free feeds commonly gains weight. Veterinary guidance for an adult cat of average weight is a measured daily ration split across several meals, not an open bowl. An owner who is not at home at lunchtime cannot deliver the middle meal.

Commercial automatic feeders solve the timing problem, but most of them depend on a cloud service and a mobile app to program the schedule. If the vendor changes the app, or the household Wi-Fi is replaced, the device becomes a plastic box. For a student project the interesting engineering question is whether the same behaviour can be produced on a microcontroller that the owner can read, repair, and reflash.

## 1.2 Purpose

This project builds a cat feeder that dispenses dry food at three fixed times per day and also on demand from a phone. The device serves its own web dashboard over the local Wi-Fi network, so no account, no cloud service, and no vendor app are required. The owner opens a browser, sees the current state of the flap and the number of meals served today, and can trigger a feeding with one tap.

The project is also a study of the boundary between software and hardware. The interesting part is not the web page. It is the electrical and timing behaviour that the web page depends on: a servo that stalls and browns out the board, a clock that is wrong until the network answers, and a flap that must never stay open.

## 1.3 Target Users and Intended Experience

The system has two users with very different needs. The design is judged on whether both are served without one damaging the other.

### 1.3.1 Persona: The owner

**Table 1.** Owner persona

| Attribute | Detail |
|---|---|
| Name | Nok |
| Age | 29 |
| Occupation | Office worker, Chiang Mai |
| Background | Lives in a rented room with one adult cat. Leaves at 07:30 and returns at 18:30. Uses a phone for everything. Has no tools beyond a screwdriver. |
| Goals | Feed the cat on a predictable schedule. Check from work that the morning meal was served. Add an extra meal when the cat begs. |
| Pain points | Cannot come home at lunch. Forgets whether the morning meal was served. Does not want another app account, and does not want to learn a configuration language. |
| Experience requirement | Open one page, read five values, tap one button. No login, no setup wizard, no pairing step. |

### 1.3.2 Persona: The cat

**Table 2.** Cat persona and constraints

| Attribute | Detail |
|---|---|
| Subject | One adult domestic cat, 4 kg |
| Behaviour | Eats at the sound of the flap. Approaches the device when it is near the bowl. |
| Constraint | The cat must not be able to open the flap with a paw, and must not be trapped by it. |
| Experience requirement | A meal arrives without the cat having to touch anything. The flap cycle is short enough not to startle the cat away from the bowl. |

## 1.4 Project Scope

The scope is set so that the work is complete and demonstrable on 4 November 2026. Features that cannot be finished and tested by 2 November are moved to Future Work rather than shipped half-built.

**Table 3.** Scope of the delivered system

| In scope | Out of scope |
|---|---|
| Scheduled feeding at three times per day using NTP time | Cloud storage and a hosted dashboard |
| On-demand feeding from a browser on the same network | A native mobile application |
| Flap status, daily meal count, last meal time and source | User accounts and authentication |
| Automatic flap close after a fixed open interval | Wet food dispensing |
| Cat presence detection with an ultrasonic sensor (target) | Camera or two-way audio |
| Portion control by weight with a load cell (target) | Support for more than one feeder or more than one cat |

The two target features are carried as requirements so the design is ready for them, but they ship only if the parts arrive and pass test by 2 November. The reason for the rule is that a requirement that is written and tested earns credit, while a half-connected sensor does not.

## 1.5 Objectives

1. Deliver food at 08:00, 13:00, and 18:00 local time, using time obtained from a network time server rather than a countdown timer.
2. Serve a dashboard from the ESP8266 itself, reachable from a phone on the same Wi-Fi network in under two seconds.
3. Close the flap automatically after every feeding, in all code paths, including the path where the browser disconnects immediately after sending the command.
4. Record the time, count, and trigger source of each meal so the owner can confirm a feeding happened.
5. Detect a cat within 60 cm and dispense a meal in under two seconds, at most once per 30 minutes.
6. Dispense a portion within 15 percent of the target weight.
7. Document the design so another student can rebuild the device from the repository alone.

# Chapter 2 Review and Analysis

## 2.1 Business and Software Review

The review below is written for a device that already exists in commercial form. The project is not trying to win on features. It is trying to win on independence, repairability, and cost.

### 2.1.1 Strengths

- **No cloud dependency.** The dashboard is served by the device. The system keeps working when the internet is down, and it works in a room with no router if the access point is provided by a phone.
- **Low cost.** The bill of materials is roughly one tenth of a mid-range commercial feeder, because there is no enclosure injection moulding, no app, and no subscription.
- **Measured portion instead of a timer.** Most low-cost feeders open a gate for a fixed time and trust gravity. A load cell turns the same mechanism into a closed loop, which is the difference between a timer and a controller.
- **Documented and repairable.** One firmware file, one wiring page, and one design document. A failed servo is a 100 baht replacement, not a warranty claim.

### 2.1.2 Weaknesses

- **Single point of failure.** If the ESP8266 reboots mid-cycle the meal is lost. There is no watchdog external to the board.
- **History lives in RAM.** A power cut erases the daily count. This is a known limitation and is listed in the requirements.
- **The enclosure is cardboard.** It is adequate for a demonstration and honest about being a prototype, but it is not cat-proof over months.
- **One cat, one hopper.** There is no identity and no separation if a second cat uses the device.

### 2.1.3 Opportunities

- **Presence-aware feeding.** The same ultrasonic sensor that measures the hopper level can detect an approaching cat, so one hardware decision serves two requirements.
- **Local-first pattern.** The firmware is a small, readable example of a device that serves its own UI, which is useful for other course projects and for the owner's own repairs.
- **Data without a database.** A short ring buffer in flash is enough to answer the owner's only question, which is whether the cat was fed today.

### 2.1.4 Threats

- **Servo current draw.** A stalled servo can pull enough current to brown out the board. This was observed during bring-up and is the main electrical risk.
- **Wi-Fi band mismatch.** The ESP8266 is 2.4 GHz only. A 5 GHz-only phone hotspot cannot be used.
- **Sensor false positives.** A reflective hopper wall can fool an infrared proximity sensor into reporting food that is not there. This is why ultrasonic ranging is chosen for the level measurement.
- **Scope pressure.** A five-day window invites feature creep. The scope rule in 1.4 exists for this reason.

## 2.2 Tool and Technology Review

Each decision below was made against at least one alternative. The rejected option is recorded so the reader can see the trade that was accepted.

**Table 4.** Technology decisions

| Decision | Chosen | Rejected | Rationale |
|---|---|---|---|
| Microcontroller | NodeMCU v3, ESP8266 | Arduino Uno with ESP-01, ESP32 | Already owned. Has Wi-Fi and a 3.3 V regulator on board. The ESP32 is better but adds cost and no requirement needs it. |
| Actuator | Servo SG90 | Stepper motor, solenoid | The servo holds an angle without power cycling and needs one PWM pin. A solenoid slams and needs a driver transistor. |
| Time source | SNTP over Wi-Fi | DS3231 real-time clock | No extra part, no battery, and the time is correct after every boot. An RTC drifts and needs a battery holder. |
| Transport | HTTP on the device | MQTT broker, Blynk | HTTP needs no broker and no account. Blynk hides the implementation behind a vendor widget, which removes the work the assessment is looking for. |
| Dashboard | Embedded HTML and JavaScript in PROGMEM | SPIFFS files, external CDN | One flash image, no filesystem, no internet dependency, and the page cannot lose its assets. |
| Presence sensing | HC-SR04 ultrasonic | FC-51 infrared proximity | Ultrasonic reports distance, which supports both presence and level. The FC-51 is binary and is fooled by a cardboard hopper wall. |
| Portion control | HX711 with 1 kg load cell | Open-loop flap timing | Timing assumes a constant flow rate. A load cell measures the actual mass, which allows the controller to correct. |

## 2.3 Risk Analysis

**Table 5.** Risk register

| ID | Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|---|
| R-01 | Servo stall browns out the board and reboots it mid-meal | High | High | Power the servo from the powerbank rail, not from 3V3. Add a 470 uF capacitor across VIN and GND. Test the worst case with servo_test.ino before integrating. |
| R-02 | Flap stays open and the hopper empties | Medium | High | The close is driven from millis() in the main loop, not from a network callback, so a dropped client cannot strand the flap. |
| R-03 | Clock is wrong or jumps backward after a resync | Medium | Medium | Ignore the schedule until the clock passes a fixed epoch. A duplicate meal inside the same minute is prevented by a minute key. |
| R-04 | Ultrasonic readings oscillate on a kibble surface | High | Medium | Sample ten times per measurement and use the median. Reject echoes outside the hopper depth. |
| R-05 | Load cell cannot be mounted rigidly on cardboard | Medium | Medium | Decide by 2 November. If the platform flexes, drop the feature and record it as Future Work. |
| R-06 | Wi-Fi in the examination room blocks the device | Low | High | The dashboard is reachable in the local network, so the demonstration does not depend on internet access. A phone hotspot is the fallback. |
| R-07 | Parts do not arrive before 30 October | Medium | High | The firmware already meets the schedule and dashboard requirements without any sensor. The sensors are additive, not load bearing. |

# Chapter 3 Product Perspective

## 3.1 Product Perspective

The device is a standalone appliance with a browser interface. It is not a component of a larger platform, and it does not report to a server. This choice is deliberate. A self-hosted device has a smaller attack surface, a smaller failure surface, and a demonstration that survives a broken internet connection.

Inside the appliance there are four responsibilities. The web layer turns HTTP requests into commands and status. The scheduler compares local time against a fixed table of meals. The actuation layer drives the servo. The state layer records what happened. The important design rule is that every feeding path, whether from the browser, the clock, or a sensor, passes through one function so that counting and safety rules cannot diverge.

## 3.2 Product Features

**Table 6.** Feature summary

| ID | Feature | Actor | Status |
|---|---|---|---|
| F1 | Scheduled feeding at 08:00, 13:00, and 18:00 using NTP time | System clock | Delivered |
| F2 | On-demand feeding from the web dashboard | Owner | Delivered |
| F3 | Flap status, daily count, last meal, and connection quality | Owner | Delivered |
| F4 | Cat presence detection and presence-triggered meal | Cat | Target |
| F5 | Portion control by weight, and hopper level | System | Target |

## 3.3 System Architecture

The context diagram below shows every entity that the device exchanges energy or data with. Dashed elements are planned and are marked as such so the diagram does not claim hardware that is not on the bench.

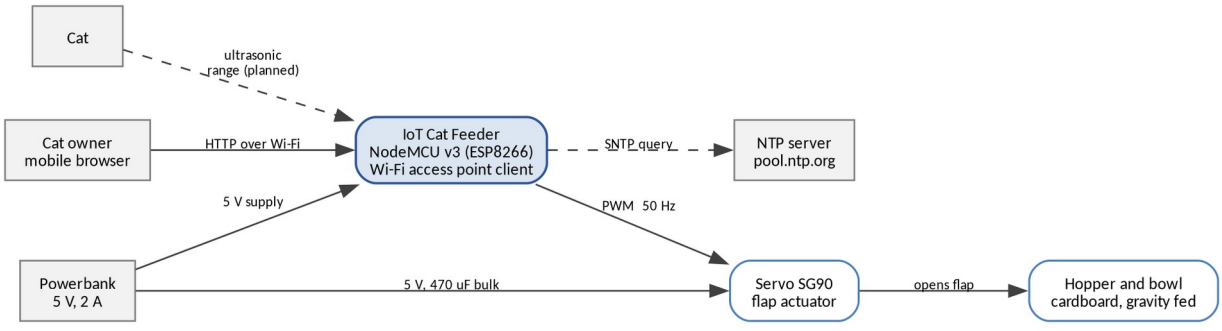



**Figure 1.** Context diagram of the proposed system.

## 3.4 Use Case Overview

Five use cases are identified. UC-01 to UC-03 are delivered. UC-04 and UC-05 are the presence and portion targets.

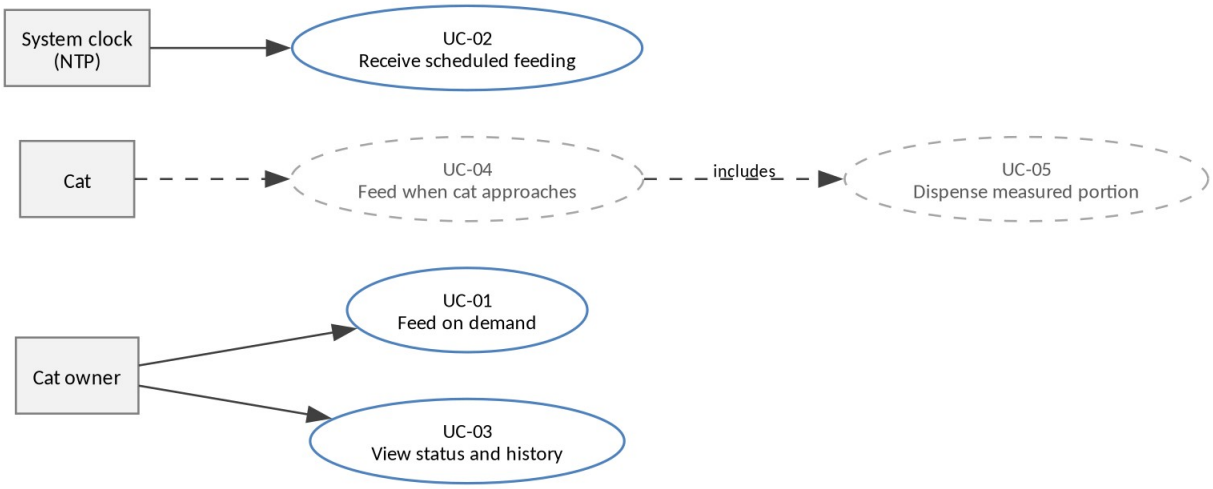



**Figure 2.** Use case diagram. Dashed use cases are planned.

# Chapter 4  Quality Planning

Quality requirements are grouped by category and given identifiers so the design and the test plan can refer to them. The full text of each requirement is in the SRS.

**Table 7.** Quality requirement summary

| Category | ID | Requirement in one line |
|---|---|---|
| Performance | NFR-PM-01 | Dashboard responds within 2 seconds on the same network. |
| Performance | NFR-PM-02 | Presence detection dispenses within 2 seconds of the cat entering range. |
| Reliability | NFR-RL-01 | Flap never remains open longer than 3 seconds. |
| Reliability | NFR-RL-02 | Device reconnects to Wi-Fi without a reboot. |
| Reliability | NFR-RL-03 | Schedule works for 24 hours without drift beyond 1 minute. |
| Usability | NFR-UH-01 | Dashboard is readable on a 375 px wide screen without zoom. |
| Usability | NFR-UH-02 | A first-time user can feed the cat without instructions. |
| Security | NFR-SEC-01 | Wi-Fi credentials are never committed to the repository. |
| Security | NFR-SEC-02 | Only the documented HTTP methods are accepted. |
| Maintainability | NFR-OP-01 | The firmware builds with the Arduino ESP8266 core and documented libraries. |
| Constraint | NFR-CP-01 | Runs from a 5 V, 2 A powerbank with no laptop attached. |
| Constraint | NFR-CP-02 | Operates at 2.4 GHz Wi-Fi only. |

# Chapter 5 Limits, Milestones, and Deliverables

## 5.1 Limits

- The submission deadline is 4 November 2026. Any feature not tested by 2 November is descoped.
- The build is limited to parts already owned plus a sensor budget of about 500 baht.
- The enclosure is built from cardboard and a food-safe bowl. Machining and 3D printing are not available in the time window.
- The device runs on a single board with no server, no database, and no cloud account.

## 5.2 Milestones

**Table 8.** Milestones

| Date | Milestone | Exit criterion |
|---|---|---|
| 30 Oct 2026 | Documents and parts order | Proposal, SRS, and SDD drafted. Sensors ordered. |
| 31 Oct 2026 | Requirements frozen | Requirement set baselined in version control. |
| 1 Nov 2026 | Presence detection integrated | Cat detected in 16 of 20 trials. |
| 2 Nov 2026 | Portion control integrated or descoped | Weight within 15 percent, or feature removed. |
| 3 Nov 2026 | System test and evidence | All test cases executed. Photos and video captured. |
| 4 Nov 2026 | Submission | Repository tagged, report submitted. |

## 5.3 Deliverable Items

1. The assembled feeder, working from a powerbank.
2. Firmware source in firmware/feeder/feeder.ino and the servo bring-up sketch in firmware/servo_test/servo_test.ino.
3. Wiring diagram in docs/wiring_nodemcu_servo.html.
4. This proposal, the SRS, and the SDD.
5. Test records with photographs and a demonstration video.
6. The output and discussion chapter with measured results.
7. The conclusion and future work chapter.

## 5.4 Budget

Prices are estimates from general retail listings and are not yet confirmed with the supplier.

**Table 9.** Estimated bill of materials

| Item | Quantity | Unit (baht) | Total (baht) |
|---|---|---|---|
| NodeMCU v3 (already owned) | 1 | 0 | 0 |
| Servo SG90 (already owned) | 1 | 0 | 0 |
| Capacitor 470 uF 16 V | 1 | 10 | 10 |
| HC-SR04P ultrasonic sensor | 2 | 80 | 160 |
| HX711 amplifier with 1 kg load cell | 1 | 160 | 160 |
| Breadboard, jumpers, resistor (already owned) | 1 set | 0 | 0 |
| Cardboard, tape, food-safe bowl | 1 set | 50 | 50 |

| Item | Quantity | Unit (baht) | Total (baht) |
|---|---|---|---|
| Powerbank 5 V 2 A (already owned) | 1 | 0 | 0 |
| **Total** | | | **380** |

## 5.5 Future Work

- Move the daily history into flash so it survives a power cut.
- Add an ESP32-CAM module for a still image per meal, so the owner can confirm the cat ate rather than the bowl.
- Identify the cat with an RFID collar tag so a second cat cannot take the portion.
- Add a hopper level alarm that publishes to the dashboard instead of only showing a value.
- Replace the cardboard enclosure with a printed or laser-cut body.

# Part II Software Requirements Specification

*Functions, use cases, interfaces, and verifiable requirements.*

# Chapter 6 Introduction

## 6.1 Purpose

This document specifies the requirements for the IoT Cat Feeder, version 1.0.0. It is written for the project advisor as the acceptance basis, for the developer as the input to the design, and for the examiner as the evidence that each requirement has a stated, checkable condition.

The document follows the structure of ISO/IEC/IEEE 29148 for software requirements. The section order is adapted to the size of the project: the standard separates product functions from detailed requirements, which this document keeps, and it separates verification from the requirement text, which this document keeps as a matrix in Section 7.10.

## 6.2 Scope

The product is a single-board appliance that dispenses dry cat food on a schedule and on demand, and reports its state through a web page served by the same board.

**Table 10.** System identification

| Item | Value |
|---|---|
| Product name | IoT Cat Feeder |
| Version specified | 1.0.0 |
| Platform | NodeMCU v3, ESP8266EX, 80 MHz, 4 MB flash |
| Runtime | Arduino core for ESP8266, single-threaded main loop |
| Network role | Wi-Fi station client with an embedded HTTP server on port 80 |
| Users | One owner, one cat |
| Operating environment | Indoor, 2.4 GHz Wi-Fi, 5 V 2 A supply |

The product does not store data outside the device, does not authenticate users, and does not depend on any external service other than a network time server.

## 6.3 Product Perspective

The device is a standalone system. It replaces the manual act of pouring food, not a software component inside another product.

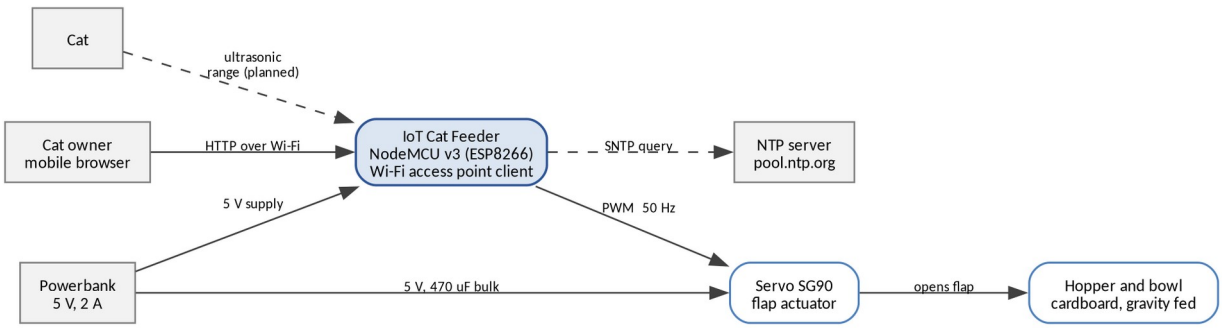



**Figure 3.** System context. Dashed elements are planned.

The owner is the only human actor. The clock is an external service that provides the time reference. The cat is an environmental actor whose position may become an input once the presence requirement is delivered.

## 6.4 Product Functions

**Table 11.** Product functions

| ID | Function | Summary | Status |
|---|---|---|---|
| F1 | Scheduled feeding | Dispenses one meal at each configured time, using network time. | Delivered |
| F2 | On-demand feeding | Dispenses one meal when the owner presses the dashboard button. | Delivered |
| F3 | Status and history | Reports flap state, daily count, last meal, and link quality. | Delivered |
| F4 | Presence-triggered feeding | Dispenses a meal when a cat is detected near the bowl. | Target |
| F5 | Portion and level control | Dispenses a measured mass and reports hopper level. | Target |

## 6.5 User Characteristics

**Table 12.** User classes

| User class | Technical level | Characteristics and needs |
|---|---|---|
| Owner | Non-technical | Uses a phone browser. Expects a single page with a single action. Cannot be asked to install software, create an account, or read a manual. |
| Maintainer | Technical | Rebuilds and reflashes the device. Needs the wiring, the pin map, the library list, and the credentials file documented. |
| Cat | None | Interacts physically. Must not be able to open the flap or be caught by it. |

## 6.6 Limitations

- History is held in RAM. A reboot clears the daily count.
- Any device on the same Wi-Fi network can issue a feeding command, because there is no authentication.
- Only dry food with a free-flowing particle size can be dispensed.
- The device cannot detect that the bowl is already full before dispensing.
- Only one feeder and one hopper are supported.
- The system does not operate without a 5 V supply.

## 6.7 Assumptions and Dependencies

**Table 13.** Assumptions and dependencies

| ID | Statement | Effect if false |
|---|---|---|
| A-01 | The Wi-Fi network is 2.4 GHz and reachable from the device position. | The device cannot join. The dashboard is unreachable. |
| A-02 | A network time server is reachable at least once after each boot. | Scheduled feeding is disabled. Manual feeding still works. |
| A-03 | The hopper holds at least 300 g of dry food. | Meals become short. The level requirement reports the shortage. |
| A-04 | The servo can lift the flap without stalling. | Brownout or a failed meal. R-01 in the proposal covers the mitigation. |
| A-05 | The owner's phone and the device are on the same subnet. | The dashboard cannot be reached by name or address. |

| ID | Statement | Effect if false |
|---|---|---|
| A-06 | The load cell can be mounted on a rigid platform. | Portion control is descoped and recorded as future work. |

## 6.8 Acronyms and Definitions

### 6.8.1 Acronyms

**Table 14.** Acronyms

| Acronym | Meaning |
|---|---|
| AP | Access point |
| GPIO | General purpose input and output |
| HTTP | Hypertext Transfer Protocol |
| HX711 | 24-bit load cell amplifier |
| JSON | JavaScript Object Notation |
| NTP, SNTP | Network Time Protocol and its simplified form |
| PWM | Pulse width modulation |
| SRS | Software Requirements Specification |
| SDD | Software Design Description |
| UART | Universal asynchronous receiver and transmitter |

### 6.8.2 Definitions

**Table 15.** Definitions

| Term | Definition |
|---|---|
| Meal | One dispense cycle, defined as one flap open event followed by an automatic close. |
| Flap | The servo-driven cover that gates the hopper outlet. |
| Hopper | The container that holds dry food above the flap. |
| Schedule minute | A minute of local time that appears in the configured meal table. |
| Source | The trigger of a meal: `manual`, `schedule`, or `presence`. |
| Minute key | An integer that identifies one calendar minute, used to prevent duplicate meals. |
| Target feature | A requirement that is specified and designed but ships only if the parts pass test before 2 November 2026. |

## 6.9 References

**Table 16.** References

| Reference | Use in this document |
|---|---|
| ISO/IEC/IEEE 29148:2018, Requirements engineering | Structure of the requirement sections. |
| IEEE Std 1016-2009, Software design descriptions | Structure of the companion SDD. |
| Arduino core for ESP8266, reference documentation | Behaviour of `ESP8266WebServer`, `configTime`, and `WiFi`. |
| HC-SR04 ultrasonic sensor datasheet | Range, beam angle, and supply voltage limits. |
| HX711 datasheet | Clock timing and gain selection. |

| Reference | Use in this document |
|---|---|
| SG90 servo datasheet | Stall current and torque limits. |
| Project rubric, Idea 50 / Circuit 30 / Doc 20 | Coverage check in Appendix A. |

## 6.10 Document Control

**Table 17.** Revision history

| Version | Date | Author | Change |
|---|---|---|---|
| 0.9 | 30 October 2026 | C. Jaiman | First draft in Thai, requirements only. |
| 1.0.0 | 4 November 2026 | C. Jaiman | Baselined. Sensor requirements marked as target. |

# Chapter 7 Specification Requirements

## 7.1 Feature Description

### 7.1.1 F1: Scheduled Feeding

At each configured meal time the device dispenses one meal without any user action. The time reference comes from a network time server, so the schedule stays correct after a power cut without a battery-backed clock.

**Table 18.** F1 parameters

| Parameter | Value |
|---|---|
| Meal times | 08:00, 13:00, 18:00 |
| Time zone | UTC+7, no daylight saving |
| Time sources | pool.ntp.org, time.nist.gov |
| Duplicate suppression | One meal per calendar minute |
| Behaviour before time is valid | Scheduler disabled, dashboard shows --:--:-- |

### 7.1.2 F2: On-Demand Feeding

The owner presses a button on the dashboard and receives a meal within 1.5 seconds. The request is a POST to /feed. The button disables itself for 1.3 seconds after the press so that a double tap does not produce two overlapping cycles.

**Table 19.** F2 parameters

| Parameter | Value |
|---|---|
| Endpoint | POST /feed |
| Response | 200 with {"ok":true} |
| Flap open duration | 900 ms |
| Client lockout | 1300 ms |
| Accepted methods | POST only. Others return 404. |

### 7.1.3 F3: Status and History

The dashboard shows five values and refreshes without user action. The values answer one question: was the cat fed today, and by what.

**Table 20.** F3 parameters

| Displayed value | Source field | Format |
|---|---|---|
| Flap state | open | Open, or Closed |
| Meals today | count | Integer with unit |
| Last meal | last, source | HH:MM with trigger in brackets |
| Device time | time | HH:MM:SS |
| Configured schedule | schedule | Three times separated by a middle dot |
| Link quality | rssi, ip | dBm and dotted quad |

### 7.1.4 F4: Presence-Triggered Feeding (target)

An ultrasonic sensor aimed at the approach path detects a cat within 60 cm. A confirmed detection dispenses one meal, subject to a cooldown and a daily cap. Confirmation requires three consecutive readings inside range, which prevents a single stray echo from feeding the cat.

**Table 21.** F4 parameters

| Parameter | Value |
|---|---|
| Detection range | 60 cm |
| Confirmation | 3 consecutive readings |
| Cooldown after a presence meal | 30 minutes |
| Daily cap | Configurable, default 6 |
| Sensor | HC-SR04 or HC-SR04P, echo divided to 3.3 V |

### 7.1.5 F5: Portion and Level Control (target)

A load cell under the bowl measures the mass dispensed. The controller closes the flap when the target mass is reached, or after a timeout, whichever comes first. A second ultrasonic sensor aimed down into the hopper estimates the remaining food.

**Table 22.** F5 parameters

| Parameter | Value |
|---|---|
| Target portion | 20 g per meal, adjustable |
| Tolerance | plus or minus 15 percent |
| Flap timeout | 3 s, then close regardless of mass |
| Level reporting | Percentage of usable hopper depth |
| Amplifier gain | Channel A, gain 128 |

## 7.2 Use Case Diagram

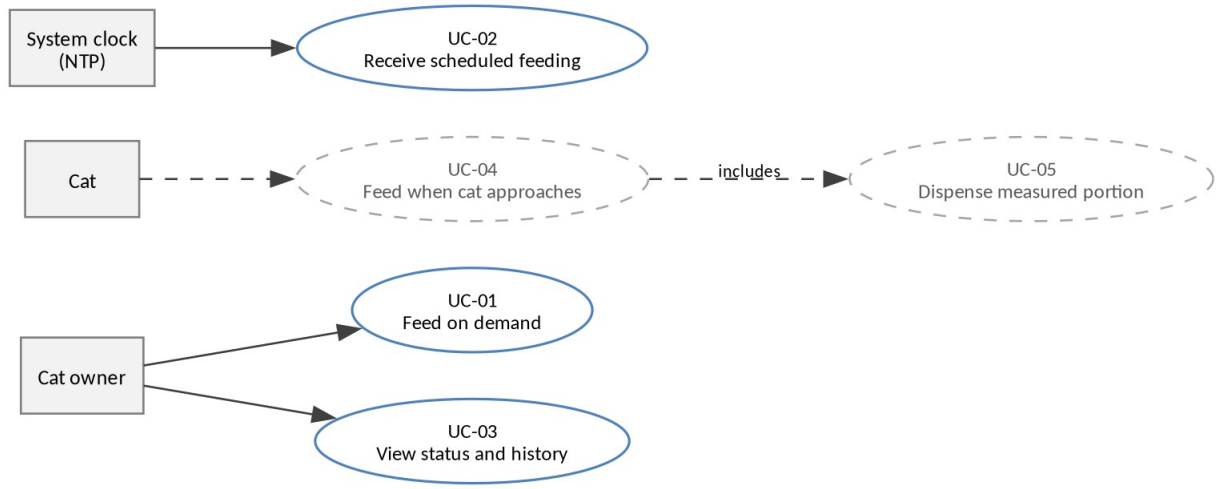



**Figure 4.** Use cases. Dashed use cases are targets and are not part of the delivered baseline.

## 7.3 Use Case Description

### 7.3.1 UC-01: Feed the Cat on Demand

**Table 23.** UC-01: Feed the cat on demand

| Item | Description |
|---|---|
| Actor | Owner |
| Goal | Dispense one meal immediately. |
| Precondition | The device is powered and reachable on the network. A browser has loaded the dashboard. |
| Trigger | The owner presses the feeding button. |
| Main flow | 1. The browser sends POST /feed.<br>2. The device validates the method.<br>3. The device records the meal with source manual.<br>4. The device drives the servo to the open angle.<br>5. The device returns {"ok":true}.<br>6. After 900 ms the main loop closes the flap. |
| Alternate flow | A1. The request times out: the browser shows a connection failure, and the flap still closes because the close is driven locally. |
| Postcondition | One meal is counted. The flap is closed. |
| Frequency | Up to several times per day. |

### 7.3.2 UC-02: Receive a Scheduled Feeding

**Table 24.** UC-02: Receive a scheduled feeding

| Item | Description |
|---|---|
| Actor | System clock, provided by NTP |
| Goal | Dispense one meal at a configured time without user action. |
| Precondition | The device has a valid time, meaning the epoch is later than 9 September 2020. |
| Trigger | Local time reaches a minute in the meal table. |
| Main flow | 1. The main loop reads the local time.<br>2. The minute key is computed.<br>3. If the key equals the previous key, the loop exits.<br>4. If the minute matches a meal time, one meal is dispensed with source schedule.<br>5. The flap closes after 900 ms. |
| Alternate flow | A1. Time is not yet valid: the loop exits without dispensing. A2. The clock jumps backward after a resync: a repeated minute key still produces at most one meal. |
| Postcondition | One meal is counted with source schedule. |
| Frequency | Three times per day. |

### 7.3.3 UC-03: View Status and History

**Table 25.** UC-03: View status and history

| Item | Description |
|---|---|
| Actor | Owner |
| Goal | Confirm that the cat has been fed and inspect the device clock. |

| Item | Description |
|---|---|
| Precondition | The device is reachable. |
| Trigger | The owner opens the dashboard, or the two-second refresh fires. |
| Main flow | 1. The browser sends GET /status.<br>2. The device serialises eight fields.<br>3. The page updates five display rows and the link footer. |
| Alternate flow | A1. The request fails: the footer reports a lost connection and the previous values stay on screen. |
| Postcondition | No state changes on the device. |
| Frequency | Continuous while the page is open. |

### 7.3.4 UC-04: Feed When the Cat Approaches (target)

**Table 26.** UC-04: Feed when the cat approaches

| Item | Description |
|---|---|
| Actor | Cat |
| Goal | Dispense a meal when the cat is at the bowl and the cooldown has expired. |
| Precondition | The presence sensor is connected and calibrated. The daily cap is not reached. |
| Trigger | Three consecutive distance readings below 60 cm. |
| Main flow | 1. The loop polls the sensor.<br>2. The median distance is computed.<br>3. The confirmation counter increments.<br>4. A meal is dispensed with source presence.<br>5. The cooldown starts. |
| Alternate flow | A1. The cat leaves before confirmation: the counter resets. A2. Cooldown is active: the request is ignored and no meal is counted. |
| Postcondition | One meal is counted. The presence cooldown is running. |
| Frequency | At most once per 30 minutes. |

### 7.3.5 UC-05: Dispense a Measured Portion (target)

**Table 27.** UC-05: Dispense a measured portion

| Item | Description |
|---|---|
| Actor | System |
| Goal | Deliver a mass of food within tolerance of the target. |
| Precondition | The load cell is calibrated at zero with an empty bowl. |
| Trigger | Any feeding path starts. |
| Main flow | 1. The flap opens.<br>2. The amplifier is sampled ten times per second.<br>3. The mass delta is compared with the target.<br>4. The flap closes when the target is reached. |
| Alternate flow | A1. The timeout of 3 s expires: the flap closes and the meal is recorded with a shortfall flag. |
| Postcondition | The dispensed mass and the shortfall are recorded. |
| Frequency | Once per meal. |

### 7.3.6 UC-06: Recover From a Network Loss

**Table 28.** UC-06: Recover from a network loss

| Item | Description |
|---|---|
| Actor | System |
| Goal | Return to the network without a reboot. |
| Precondition | The device was connected and lost the link. |
| Trigger | The connection state changes to disconnected. |
| Main flow | 1. The loop checks the connection state.<br>2. A retry is issued no more than once every 5 seconds.<br>3. On success the local address is printed once to the serial console. |
| Alternate flow | A1. The network is absent: retries continue indefinitely at the same rate. |
| Postcondition | The dashboard is reachable again. No meal state is lost. |
| Frequency | On every disconnection. |

## 7.4 Activity Diagram

The activity diagram below shows one iteration of the main loop. The close of the flap is evaluated on every iteration and does not depend on an HTTP request completing.

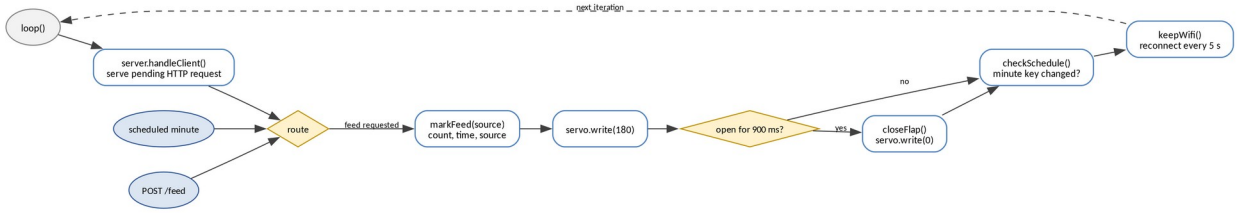



**Figure 5.** Main loop activity.

## 7.5 Functional Requirements

Priority uses MoSCoW. Verification uses one of T for test, D for demonstration, A for analysis, or I for inspection.

**Table 29.** Functional requirements

| ID | Requirement | Priority | Verification | Status |
|---|---|---|---|---|
| FR-01 | The device shall dispense one meal at 08:00, 13:00, and 18:00 local time. | Must | T | Delivered |
| FR-02 | The device shall obtain local time from a network time server and shall not evaluate the schedule until the time is valid. | Must | T | Delivered |
| FR-03 | The device shall dispense at most one scheduled meal per calendar minute. | Must | T | Delivered |
| FR-04 | The device shall dispense one meal when it receives POST /feed. | Must | T | Delivered |
| FR-05 | The device shall close the flap within 1000 ms of opening it, on every code path. | Must | T | Delivered |
| FR-06 | The device shall reject any method other than POST on /feed. | Must | T | Delivered |
| FR-07 | The device shall return the eight status fields defined in the status table above on GET /status. | Must | T | Delivered |

| ID | Requirement | Priority | Verification | Status |
|---|---|---|---|---|
| FR-08 | The dashboard shall refresh its values every 2 seconds without user action. | Must | D | Delivered |
| FR-09 | The device shall reset the daily meal count when the local date changes. | Must | T | Delivered |
| FR-10 | The device shall record the trigger source of every meal as manual, schedule, or presence. | Must | T | Delivered |
| FR-11 | The device shall serve the dashboard page on GET / with no external asset. | Must | T | Delivered |
| FR-12 | The device shall return 404 for every undefined path. | Must | T | Delivered |
| FR-13 | The device shall detect a cat within 60 cm of the bowl after three consecutive readings. | Should | T | Target |
| FR-14 | The device shall not dispense more than one presence-triggered meal in 30 minutes. | Should | T | Target |
| FR-15 | The device shall dispense a mass within 15 percent of the configured target. | Could | T | Target |
| FR-16 | The device shall report hopper level as a percentage of usable depth. | Could | D | Target |
| FR-17 | The device shall persist the daily history across a reboot. | Could | T | Future work |
| FR-18 | The device shall refuse a meal when the configured daily cap is reached. | Should | T | Future work |

## 7.6 External Interface Requirements

### 7.6.1 User Interface

One page, one column, maximum width 420 px, in Thai. The page uses a light background and a single accent colour. It has no navigation, no form, and no settings panel.

**Table 30.** Dashboard elements

| Element | Type | Behaviour |
|---|---|---|
| Title block | Static text | Names the device and states the primary action. |
| Status card | Five text rows | Rebuilt from /status on every refresh. |
| Feed button | Button | Sends POST /feed, disables itself for 1300 ms. |
| Link footer | Text | Shows RSSI and IP, or a lost-connection message. |

### 7.6.2 Software Interface

**Table 31.** HTTP interface

| Method | Path | Request body | Response |
|---|---|---|---|
| GET | / | None | HTML, 200 |
| GET | /status | None | JSON, 200, application/json; charset=utf-8 |
| POST | /feed | None | JSON {"ok":true}, 200 |
| Any | Other | None | Text, 404 |

**Table 32.** Status response fields

| Field | Type | Range | Meaning |
|---|---|---|---|
| open | Boolean | true, false | Flap is open. |
| count | Integer | 0 to 255 | Meals counted since local midnight. |
| last | String | HH:MM or - | Local time of the last meal. |
| source | String | manual, schedule, presence, empty | Trigger of the last meal. |
| time | String | HH:MM:SS or --:--:-- | Device local time. |
| schedule | String | Three times | Configured meal times. |
| ip | String | Dotted quad | Device address. |
| rssi | Integer | -90 to -30 | Signal strength in dBm. |

### 7.6.3 Hardware Interface

**Table 33.** Hardware interface

| Device | Signal | Pin | Electrical notes | Status |
|---|---|---|---|---|
| Servo SG90 | PWM | D5, GPIO14 | 50 Hz, 1 to 2 ms pulse. Powered from VIN, not 3V3. | Connected |
| Servo SG90 | Supply | VIN, GND | 5 V from powerbank. 470 uF across the rail. | Connected |
| HC-SR04 | Trig | To be assigned | 3.3 V drive is accepted. | Target |
| HC-SR04 | Echo | To be assigned | 5 V output requires a divider for 3.3 V logic. | Target |
| HX711 | DT, SCK | To be assigned | Two digital lines. Avoid GPIO0, GPIO2, GPIO15 at boot. | Target |
| Status LED | Digital out | D4, GPIO2 | Bring-up only. Active low on this board. | Test |

## 7.7 Data Requirements

The device keeps one record in RAM. There is no database, no file system, and no remote store in the delivered baseline.

**Table 34.** Persistent and in-memory data

| Name | Type | Lifetime | Purpose |
|---|---|---|---|
| Feeder state | Struct of six fields | Until reboot | Flap state, count, last meal, source, day. |
| Meal table | Constant array of three hours | Firmware constant | Schedule definition. |
| Minute key | Integer | Until reboot | Duplicate suppression for the scheduler. |
| Last connection attempt | Unsigned long | Until reboot | Rate limit for Wi-Fi retries. |

Requirement FR-17 would add a flash record with one line per meal, written append-only, capped at the last 32 meals. That record is not part of version 1.0.0.

## 7.8 Quality Requirements

**Table 35.** Non-functional requirements

| ID | Category | Requirement | Verification |
|---|---|---|---|
| NFR-PM-01 | Performance | A dashboard request completes within 2 s on the same network. | T |
| NFR-PM-02 | Performance | A manual meal starts within 1.5 s of the button press. | T |
| NFR-PM-03 | Performance | Presence detection dispenses within 2 s of entering range. | T |
| NFR-RL-01 | Reliability | The flap is never open longer than 3 s. | T |
| NFR-RL-02 | Reliability | The device rejoins the network within 30 s of it returning, with no reboot. | T |
| NFR-RL-03 | Reliability | Schedule timing drifts by no more than 1 minute over 24 hours. | T |
| NFR-RL-04 | Reliability | A failed HTTP response does not leave the flap open. | A |
| NFR-UH-01 | Usability | The dashboard is readable at 375 px width without zoom. | D |
| NFR-UH-02 | Usability | A first-time user feeds the cat without instruction. | D |
| NFR-UH-03 | Usability | The dashboard states the connection state when the link fails. | D |
| NFR-SEC-01 | Security | Wi-Fi credentials are excluded from version control. | I |
| NFR-SEC-02 | Security | Only documented methods and paths are served. | T |
| NFR-SEC-03 | Security | The device exposes no administrative endpoint. | I |
| NFR-OP-01 | Maintainability | The firmware builds from a documented library set. | I |
| NFR-OP-02 | Maintainability | The pin map and wiring are documented in one page. | I |
| NFR-CP-01 | Constraint | The device runs from a 5 V 2 A powerbank with no host computer. | T |
| NFR-CP-02 | Constraint | The device operates on 2.4 GHz Wi-Fi only. | I |
| NFR-CP-03 | Constraint | The firmware fits in the 4 MB flash of the NodeMCU v3. | I |

## 7.9 Design Constraints

- The servo signal uses GPIO14 because the board exposes it as D5 with no boot-time conflict.
- GPIO16 is not usable for the load cell clock because it has no interrupt support on this core.
- The echo line of a 5 V ultrasonic sensor must be divided before it reaches a 3.3 V input.
- No blocking delay is permitted in the main loop, because the HTTP server is polled from the same loop.
- The dashboard must be complete in one response under 4 KB, because it is compiled into the binary.

## 7.10 Verification

Each requirement maps to one test case in the SDD. The matrix below states the method and the pass condition at requirement level.

**Table 36.** Verification matrix

| Requirement group | Method | Pass condition |
|---|---|---|
| FR-01 to FR-03 | Timed observation | Exactly three scheduled meals in 24 hours, each within 1 minute. |
| FR-04, FR-05 | Manual test, 50 repetitions | Every repetition dispenses and closes. No flap stays open. |
| FR-06, FR-12 | Request inspection | Non-POST and unknown paths return 404 with no side effect. |
| FR-07, FR-10, FR-11 | Response inspection | All fields present, values match the device state. |

| Requirement group | Method | Pass condition |
|---|---|---|
| FR-08 | Observation | The page changes without user action within 2 s. |
| FR-09 | Clock rollover | The count returns to zero after local midnight. |
| FR-13, FR-14 | Field test with the cat | 16 of 20 detections correct. No second meal inside 30 minutes. |
| FR-15, FR-16 | Scale comparison | Dispensed mass within 15 percent of target on 10 trials. |
| FR-17 | Power cycle | History survives a reboot. Currently fails by design. |
| NFR-PM-01 | Stopwatch | Median response under 2 s across 20 requests. |
| NFR-RL-02 | Router power cycle | The device rejoins with no manual action. |
| NFR-SEC-01 | Repository inspection | secrets.h is absent from the tree and from history. |

# Appendix A Rubric Coverage

The assessment rubric weights Idea 50, Circuit 30, and Doc 20. The table maps each criterion to the section that satisfies it.

**Table 37.** Rubric coverage

| Criterion | Weight | Where it is answered |
|---|---|---|
| Concept and problem fit | Idea | Proposal, motivation and purpose |
| Innovation and impact | Idea | Proposal, strengths and opportunities; feature F5 |
| UX and use-case clarity | Idea | Proposal, target users; use case descriptions; dashboard layout |
| Architecture concept and feasibility | Idea | Proposal, technology review; design document, architecture chapter |
| Hardware implementation | Circuit | Hardware interface table; design document, power design |
| Software and firmware | Circuit | Functional requirements; design document, component description |
| System integration and IoT connectivity | Circuit | HTTP interface; design document, interface design |
| SRS | Doc | This document |
| Proposal and abstract | Doc | Proposal, introduction |
| SDD | Doc | Design document, architecture and detailed design |
| Output and discussion | Doc | To be written after testing on 3 November |
| Photos and evidence | Doc | Test record and demonstration video |
| Conclusion and future work | Doc | Proposal, future work; conclusion to be written |

# Part III Software Design Description

*Architecture, detailed design, traceability, and the test plan.*

# Chapter 8 Introduction

## 8.1 Purpose

This document describes how the IoT Cat Feeder is built. It is the design counterpart to the Software Requirements Specification, and it is organised around the design views named in IEEE Std 1016: decomposition, dependency, interface, and detail. Every design entity below traces back to a requirement, and every requirement traces forward to a test.

The reader this document serves is the next student who has to rebuild the device, or the examiner who has to decide whether the code and the circuit are one system or two unrelated artefacts.

## 8.2 Scope

The design covers the firmware in firmware/feeder/feeder.ino, the wiring in docs/wiring_nodemcu_servo.html, and the power arrangement. It excludes the mechanical construction of the cardboard hopper beyond the flap geometry, which is described only where it constrains the servo.

## 8.3 Design Overview

The firmware is one translation unit with no classes, no threads, and no dynamic allocation after start-up. It runs a single cooperative loop that serves HTTP, watches the clock, and closes the flap. Each of those three duties is a function call, and none of them blocks.

The single organising rule is that all feeding passes through feed(source). Three triggers converge on it: the HTTP handler, the scheduler, and the presence detector. Counting, safety, and the daily cap therefore live in one place instead of three.

## 8.4 Definitions

**Table 38.** Design terms

| Term | Meaning in this document |
|---|---|
| Component | A named function or small group of functions with one responsibility. |
| State layer | The struct Feeder instance and the functions that read or write it. |
| Minute key | Integer (day * 100 + hour) * 100 + minute, used to detect a new minute. |
| Flap cycle | One open command followed by the automatic close. |
| Target feature | Designed here, implemented only if the parts pass test before 2 November 2026. |

# Chapter 9 System Architecture

## 9.1 Architectural Overview

The system has three layers. The network layer terminates HTTP and converts requests into commands. The control layer decides whether a meal should happen. The device layer drives the servo and reports electrical state. The layers are functions in one file rather than modules in separate files, because the whole program is 259 lines and a file split would add build complexity without adding clarity.

**Table 39.** Layer responsibilities

| Layer | Components | Responsibility |
|---|---|---|
| Network | keepWifi, server, request handlers | Maintain the link, parse requests, serialise responses. |
| Control | feed, checkSchedule, markFeed, presence detector (target) | Decide when a meal occurs and record it exactly once. |
| Device | Servo, closeFlap, sensor driver (target) | Drive the actuator and sample the sensors. |
| Presentation | PAGE, dashboard script | Render state and send one command. |

## 9.2 Context View

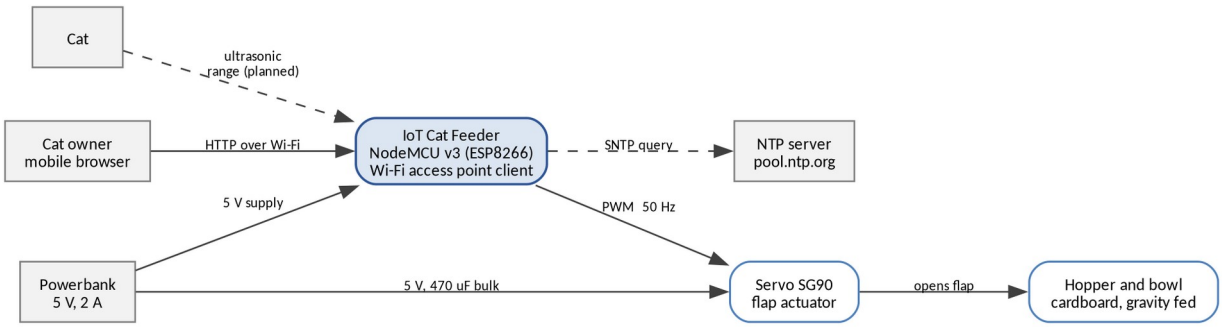



**Figure 6.** Context view: external entities and energy flow.

## 9.3 Component View

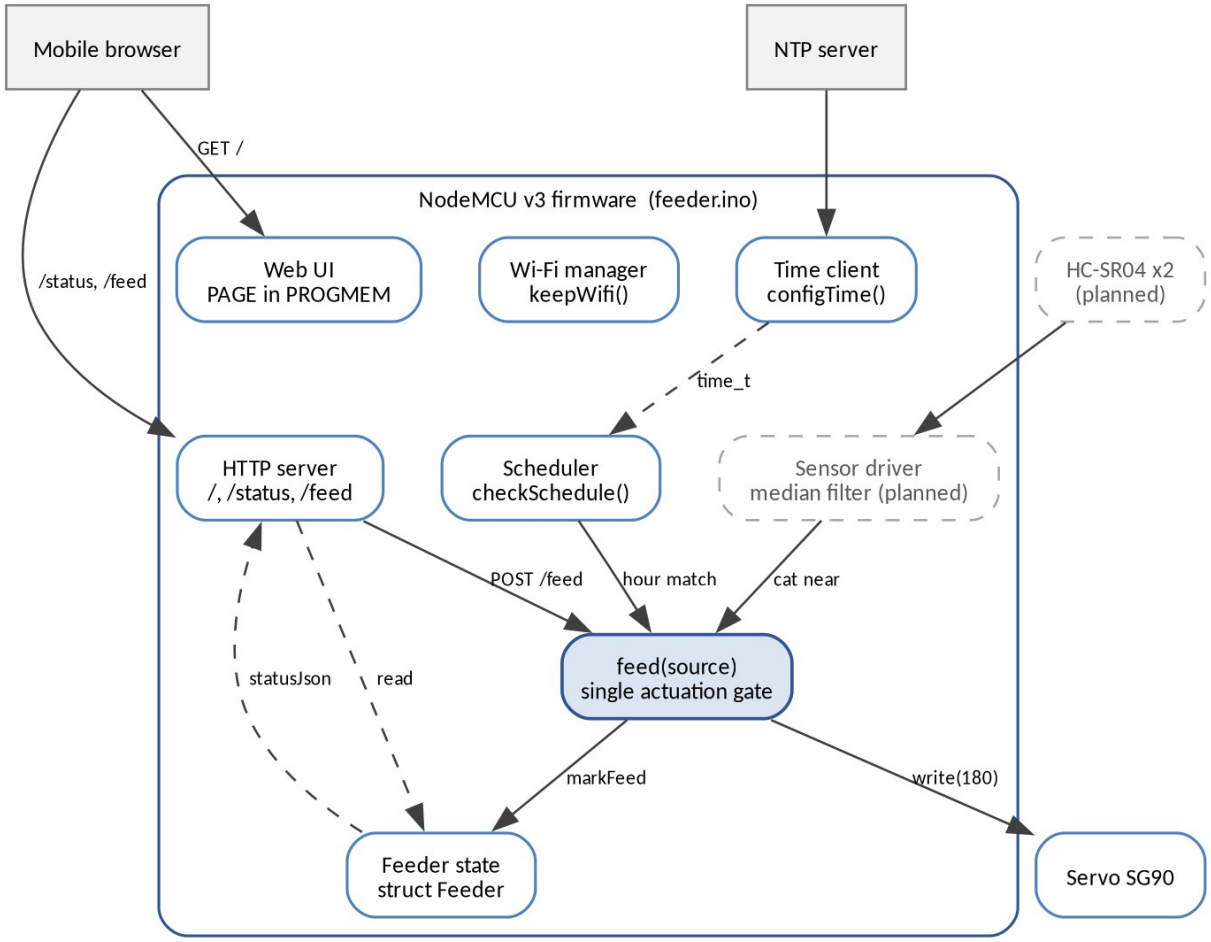



**Figure 7.** Component view. Dashed components are target features.

**Table 40.** Component catalogue

| Component | Kind | Depends on | Status |
|---|---|---|---|
| PAGE | Constant string in PROGMEM | Nothing | Delivered |
| server | ESP8266WebServer instance | PAGE, statusJson, feed | Delivered |
| statusJson | Function | Feeder, clockText, scheduleText, WiFi | Delivered |
| feed | Function | markFeed, Servo, millis | Delivered |
| closeFlap | Function | Servo, Feeder | Delivered |
| markFeed | Function | time, Feeder | Delivered |
| checkSchedule | Function | time, Feeder, feed | Delivered |
| clockText, scheduleText | Functions | time, constant table | Delivered |
| keepWifi | Function | WiFi, millis | Delivered |
| Presence detector | Function, target | Ultrasonic driver, feed | Target |
| Mass controller | Function, target | HX711 driver, feed | Target |

## 9.4 Deployment and Power Design

The device has one power input and one ground reference. The servo is the only load that draws significant current, and it is the reason the power design differs from the obvious one.

**Table 41.** Power distribution

| Rail | Source | Loads | Notes |
|---|---|---|---|
| 5 V | Powerbank, 2 A | VIN of the board, servo V+ | Servo must not be fed from 3V3. |
| 3.3 V | On-board regulator | ESP8266 core, logic | Limited to a few hundred milliamps. |
| Ground | Common | Board, servo, sensors | A single ground reference for PWM to be valid. |
| Bulk capacitance | 470 uF, 16 V | Across 5 V and ground | Absorbs the servo inrush. |

The failure that forced this design was observed during bring-up. With the servo powered from the notebook USB port, the board reset whenever the servo started. The stall current of the servo pulls the rail below the regulator dropout. Two changes fixed it: a powerbank with a higher current limit, and a 470 uF capacitor next to the servo connector. The bring-up sketch firmware/servo_test/servo_test.ino exists so that this behaviour can be reproduced without the rest of the firmware.

## 9.5 Design Rationale

**Table 42.** Design decisions and alternatives

| Decision | Alternative | Why the alternative lost |
|---|---|---|
| Single file, functions only | Split modules with headers | 259 lines. A split adds include order problems for no gain. |
| Cooperative loop, no delay | delay(900) after opening the flap | delay blocks the HTTP server. The page would freeze on every meal. |
| Close driven by millis() in the loop | Close driven by the HTTP handler | The handler runs only if the client stays connected. The loop always runs. |
| One feed(source) gate | Separate functions per trigger | Three triggers would need three copies of the counting and cap logic. |
| Embedded HTML in PROGMEM | SPIFFS file, CDN assets | No file system to corrupt, no network fetch, one flash image. |
| Boolean state for the flap | Enum state machine | Two states in the delivered scope. The enum becomes justified when the target features land. |
| In-RAM history | Flash record, external database | The owner asks one question per day. A flash record is scheduled, a database is not justified. |

# Chapter 10 Detailed Design

## 10.1 Data Design

The program holds one state record and three constants. There is no heap use after start-up, apart from the String growth inside the JSON builder, which is bounded by the buffer size.

**Table 43.** State record

| Field | Type | Initial | Meaning |
|---|---|---|---|
| open | bool | false | Flap is currently open. |
| openedAt | unsigned long | 0 | millis() at the last open command. |
| countToday | int | 0 | Meals since the last local date change. |
| lastDay | int | -1 | Day of month that countToday belongs to. |
| lastTime | String | - | HH:MM of the last meal. |
| lastSource | String | empty | Trigger of the last meal. |

**Table 44.** Constants

| Name | Value | Used by |
|---|---|---|
| SERVO_PIN | D5, GPIO14 | setup |
| CLOSED_DEG | 0 | closeFlap, setup |
| OPEN_DEG | 180 | feed |
| OPEN_MS | 900 | loop |
| SCHEDULE_HOUR | 8, 13, 18 | checkSchedule, scheduleText |
| SCHEDULE_MINUTE | 0 | checkSchedule |
| TZ_OFFSET | 25200 s | setup |

OPEN_DEG is the commanded angle, not the delivered angle. The servo in use reaches about 95 degrees before the flap linkage stops it against the hopper. The mechanism, not the code, is the real limit. A different servo requires this value to be re-measured.

## 10.2 Component Description

**Table 45.** Function catalogue

| Function | Signature | Purpose |
|---|---|---|
| setup | void setup() | Attach the servo, close the flap, start Wi-Fi, start the time client, register routes, start the server. |
| loop | void loop() | Poll HTTP, enforce the close deadline, check the schedule, keep the link. |
| feed | void feed(const String &source) | Record the meal, open the flap, stamp the open time. |
| closeFlap | void closeFlap() | Command the closed angle and clear the open flag. |
| markFeed | void markFeed(const String &source) | Update count, time, source, and handle the date change. |
| checkSchedule | void checkSchedule() | Compare local time with the meal table, at most once per minute. |
| clockText | String clockText() | Format the local time, or a placeholder before NTP. |
| scheduleText | String scheduleText() | Format the meal table for the dashboard. |

| Function | Signature | Purpose |
|---|---|---|
| statusJson | String statusJson() | Build the eight-field status object. |
| keepWifi | void keepWifi() | Reconnect at most once every 5 s, with one console line. |

### 10.2.1 Algorithm: the minute key

The scheduler must fire once per scheduled minute, but loop runs thousands of times per second. The design collapses the date and time into one integer and compares it with the value from the previous iteration.

key = (tm_mday * 100 + tm_hour) * 100 + tm_min

- A repeated call inside the same minute produces the same key, so the function exits before the meal table is examined.
- The key changes at a minute boundary, at an hour boundary, and at a month boundary, because the day field is included. Two identical keys therefore indicate the same minute, not merely the same clock face.
- The design trades a rare false negative for a common false positive. A clock that jumps backward one minute re-enters a previous key and is suppressed. A clock that jumps backward by hours may re-enter a key that matches a meal time, which is the residual risk recorded in Section 10.7.

### 10.2.2 Algorithm: median filtering for ranging (target)

A single ultrasonic reading on a kibble surface is unreliable because the beam scatters. The target design takes ten samples, sorts them, and uses the median. A detection is confirmed only after three consecutive medians fall inside the range window, which removes one-off echoes from the hopper wall or the cat's own movement.

## 10.3 State Design

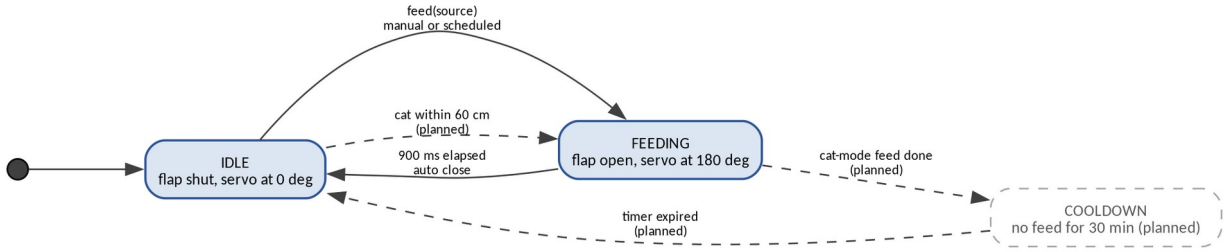



**Figure 8.** State design. Solid transitions are delivered, dashed are targets.

**Table 46.** State transitions

| From | Event | Guard | To | Action |
|---|---|---|---|---|
| IDLE | Boot | None | IDLE | Servo to closed angle. |
| IDLE | feed(source) | None | FEEDING | Record meal, servo to open angle, stamp time. |
| FEEDING | Loop iteration | millis() - openedAt >= 900 | IDLE | Servo to closed angle. |
| FEEDING | Loop iteration | Close deadline not reached | FEEDING | None. |
| IDLE | Cat within 60 cm, three times | Cooldown expired, cap not reached | FEEDING | Record meal with source presence. |
| FEEDING | Presence meal completed | None | COOLDOWN | Start 30 minute timer. |
| COOLDOW | Loop iteration | 30 minutes elapsed | IDLE | None. |

| From | Event | Guard | To | Action |
|---|---|---|---|---|
| N | | | | |

## 10.4 Interaction Flows

### 10.4.1 Flow: manual feeding

1. The dashboard sends POST /feed with an empty body.
2. server.handleClient() in the main loop dispatches the request to the registered handler.
3. The handler calls feed("manual") and sends {"ok":true}.
4. feed calls markFeed, which reads time(), formats HH:MM, resets the count if the day changed, increments the count, and stores the source.
5. feed writes the open angle and records millis().
6. On a later loop iteration, the close condition becomes true and closeFlap writes the closed angle.
7. The dashboard, polling every 2 seconds, reads open: false and the updated count.

### 10.4.2 Flow: scheduled feeding

1. A loop iteration calls checkSchedule.
2. The function reads time(). If the value is below the validity epoch, it returns.
3. It computes the minute key and compares it with the stored key. If equal, it returns.
4. It stores the new key and compares the hour and minute with each row of the meal table.
5. On a match it calls feed("schedule") and returns, so a second match in the same iteration cannot occur.

### 10.4.3 Flow: network recovery

1. Each loop iteration calls keepWifi.
2. If the link is up and the address has not been printed since the last recovery, the address is printed once.
3. If the link is down, the function returns unless 5 seconds have passed since the last attempt.
4. It starts a connection attempt and stamps the time. The attempt does not block.

## 10.5 Interface Design

### 10.5.1 HTTP contract

**Table 47.** Endpoint contracts

| Endpoint | Method | Success | Failure |
|---|---|---|---|
| / | GET | 200, HTML page | 404 for other methods |
| /status | GET | 200, JSON, eight fields | 404 for other methods |
| /feed | POST | 200, {"ok":true} | 404 for other methods |
| Any other path | Any | Not applicable | 404, plain text |

### 10.5.2 Response format

The status object is built with snprintf into a 260 byte buffer. The buffer is adequate for the current field lengths and has no overflow check. If a source string grows, the tail of the object is truncated silently. This is recorded as a limitation rather than fixed, because the source values are compile-time constants.

### 10.5.3 User interface

**Table 48.** Dashboard layout

| Region | Content | Interaction |
|---|---|---|
| Header | Device name and one-line purpose | None |
| Status card | Five rows: flap, meals today, last meal, device time, schedule | Read only |
| Action | Full-width feeding button | POST /feed, then a 1300 ms lockout |
| Footer | RSSI and IP, or a lost-connection message | None |

## 10.6 Timing and Resource Budget

The values below are a design budget, not measurements. The measured columns are filled during the test on 3 November.

**Table 49.** Timing budget

| Path | Budget | Reason |
|---|---|---|
| HTTP request to response | 40 ms | One snprintf and one send on a local network. |
| Button press to flap moving | 150 ms | Network round trip plus servo dead band. |
| Servo travel, closed to open | 200 ms | About 95 degrees at SG90 rated speed. |
| Flap open dwell | 900 ms | Fixed by OPEN_MS, long enough for a portion to fall. |
| Total flap cycle | 1.1 to 1.3 s | Travel plus dwell plus close travel. |
| Dashboard refresh | 2000 ms | setInterval in the page. |
| Schedule detection delay | Under 1 s | The loop interval bounds the check. |
| Main loop period | Under 5 ms | Four non-blocking calls. |

## 10.7 Error Handling and Recovery

**Table 50.** Failure modes

| Failure | Detection | Response | Residual risk |
|---|---|---|---|
| Wi-Fi lost | WiFi.status() | Retry every 5 s without rebooting | Dashboard unavailable until the link returns |
| Time not yet valid | Epoch below 1600000000 | Scheduler disabled, placeholder clock shown | A meal during the first seconds after boot is skipped |
| Clock jumps backward | Repeated minute key | The meal in that minute is suppressed | A jump of hours can re-enter a meal minute |
| Client disconnects mid-request | Handler never returns | The loop closes the flap on schedule | The count is already incremented |
| Servo stalls | Not detected | None in firmware | Brownout, reboot, lost meal |
| Unknown path | onNotFound | 404 with no state change | None |
| Buffer truncation | Not detected | None | Malformed JSON if a field grows |

## 10.8 Security Considerations

- The trust boundary is the Wi-Fi network. Any client on it can dispense food. This is accepted for a single household and is documented in the SRS limitations.
- No administrative endpoint exists, so there is no credential to steal from the device.
- The Wi-Fi credentials live in `secrets.h`, which is excluded by `.gitignore`. The repository ships `secrets.example.h` only.
- The device sends no telemetry outbound except the NTP query, which carries no user data.

# Chapter 11 Traceability and Verification

## 11.1 Requirements Traceability Matrix

**Table 51.** Traceability

| Requirement | Component | Test |
|---|---|---|
| FR-01, FR-02, FR-03 | checkSchedule, configTime | TC-04, TC-05 |
| FR-04, FR-05 | feed, closeFlap, loop | TC-02, TC-03 |
| FR-06, FR-12 | Route registration, onNotFound | TC-08 |
| FR-07, FR-10, FR-11 | statusJson, PAGE | TC-01, TC-06 |
| FR-08 | Dashboard script | TC-01 |
| FR-09 | markFeed | TC-07 |
| FR-13, FR-14 | Presence detector, target | TC-09 |
| FR-15, FR-16 | Mass controller, target | TC-10 |
| FR-17, FR-18 | Not implemented | Future work |
| NFR-RL-01 | loop close condition | TC-03 |
| NFR-RL-02 | keepWifi | TC-11 |
| NFR-SEC-01 | .gitignore | Inspection |

## 11.2 Test Plan

**Table 52.** Test cases

| ID | Case | Procedure | Pass condition |
|---|---|---|---|
| TC-01 | Dashboard loads | Open the device address on a phone. | Five rows populated within 2 s. |
| TC-02 | Manual feeding | Press the button, observe the flap. | Flap opens within 1.5 s. |
| TC-03 | Automatic close | Press 50 times with 5 s between presses. | All 50 close without intervention. No flap stays open. |
| TC-04 | Schedule fires | Set the next meal one minute ahead and wait. | One meal at the minute, recorded with source schedule. |
| TC-05 | Duplicate suppression | Observe the same minute for 60 s. | Exactly one meal. |
| TC-06 | Status accuracy | Compare every field with the device state. | All eight fields match. |
| TC-07 | Date rollover | Let the device run past local midnight. | Count returns to zero. |
| TC-08 | Method rejection | GET /feed, POST /status, and an unknown path. | 404 for each, with no meal dispensed. |
| TC-09 | Presence detection | Walk past the sensor 20 times at 50 to 70 cm. | At least 16 detections, no meal inside the cooldown. |
| TC-10 | Portion mass | Dispense 10 meals onto a scale. | Each within 15 percent of target. |
| TC-11 | Network recovery | Power cycle the router. | The device rejoins within 30 s, no reboot. |

## 11.3 Known Limitations

**Table 53.** Limitations accepted in version 1.0.0

| ID | Limitation | Impact | Planned fix |
|---|---|---|---|
| L-01 | History is in RAM | A reboot clears the count | Flash record, FR-17 |

| ID | Limitation | Impact | Planned fix |
|---|---|---|---|
| | | | |
| L-02 | No authentication | Any client on the network can feed | Token, or network isolation only |
| L-03 | Count increments before the servo moves | A failed servo still counts a meal | Count after the close, or verify with the load cell |
| L-04 | Status buffer is not overflow checked | Possible truncated JSON | Larger buffer or a structured writer |
| L-05 | No external watchdog | A hang stops feeding until the power is cycled | Hardware watchdog timer |
| L-06 | Single hopper, single cat | No identity or separation | RFID collar tag |

## 11.4 Future Work

- Persist the last 32 meals to flash with an append-only record.
- Add an ESP32-CAM module and store one still image per meal on the device.
- Identify the cat with an RFID collar tag so a second cat cannot take the portion.
- Add a hardware watchdog so a firmware hang recovers without the owner.
- Replace the cardboard body with a laser-cut enclosure that fixes the servo geometry.
- Publish state to an MQTT broker when the deployment network allows port 1883.

# Appendix B  Design Review Checklist

**Table 54.** Design review

| Check | Result |
|---|---|
| Every requirement maps to a component | Yes, Section 11.1 |
| Every component has one responsibility | Yes, Section 9.3 |
| No blocking call in the main loop | Yes, delay is used only in the bring-up sketch |
| The flap has a bounded open time in every path | Yes, the loop enforces it independent of HTTP |
| The failure of a client does not change device state | Yes, Section 10.7 |
| Electrical limits of each pin are respected | Partly. The echo divider is required for the target sensors and is recorded in the SRS limitations |
| The design states what it does not do | Yes, Section 11.3 |